

Building Shared State in EDSL

A guide from first principles

Shared state is data that several EDSL interviews can read and change. It makes group voting, bargaining, discussion boards, forecasting, auctions, and common task pools possible.

The precise execution contract is maintained separately in `docs/shared_state_semantics/`. That specification is normative when an example in this teaching guide is ambiguous.

This guide constructs one shared data structure explicitly. Nothing is hidden behind a recipe factory. Once every part is understood, we add grouping, surveys, storage, and convenience constructors.

1. The mental model

A shared-state system has four distinct layers:

Machine definition

↓

SharedState space

↓

SharedStateMap of spaces

↓

Execution store

A **Machine** defines one kind of data structure: what it stores, which writes are legal, and what readers can see.

A **SharedState** is one concrete space containing one or more named machines. For example, the space for `family-1` might contain a poll and a message board.

A **SharedStateMap** maps scope keys such as `family-1` and `family-2` to independent **SharedState** spaces.

A **store** persists spaces while interviews execute. It may be managed locally or remotely. Storage is separate from the data definitions.

We will build these layers in that order.

2. Example: choosing an activity

Four people will choose among:

```
activities = [
    "bike ride",
    "sailing",
    "hike",
    "beach day",
]
```

We want one vote per person, represented by a dictionary:

```
{
    "Amina": "hike",
    "Boris": "sailing",
}
```

Keys are voter names. Values must be one of the four activities. A voter's first vote wins; retries or later attempts do not replace it.

That plain-language description is the specification we will encode.

3. Constructing the Machine

Import the small shared-state language:

```

from edsl.sharedstate.dsl import (
    Machine,
    Command,
    T,
    state_field,
    constant,
    field,
    input_,
    put,
)

```

Define the complete machine:

```

activity_poll = Machine(
    name="ActivityPoll",

    constants={
        "activities": activities,
    },

    fields={
        "votes": state_field(
            T.map(
                T.text(),
                T.choice(constant("activities")),
            ),
            initial={},
        ),
    },

    commands={
        "vote": Command(
            inputs={
                "voter": T.text(),
                "activity": T.choice(constant("activities")),
            },
            effects=(
                put(
                    "votes",
                    key=input_("voter"),
                    value=input_("activity"),
                    once=True,
                ),
            ),
        ),
    },

    view={
        "votes": field("votes"),
        "vote_count": field("votes").length(),
    },
)

```

This definition contains no model calls, database connection, Python callback, or survey. It is serializable data describing a state machine.

4. Constants

Constants configure a machine but cannot be changed by commands:

```
constants={
    "activities": activities,
}
```

Refer to a constant with:

```
constant("activities")
```

This expression means “use the configured `activities` value when the machine executes.” It does not immediately retrieve a Python value. Explicit references can be validated and serialized.

Constants commonly contain choices, stakes, capacities, participant counts, round counts, payoff tables, and mechanism settings.

5. Fields

Fields persist between commands:

```
"votes": state_field(
    T.map(
        T.text(),
        T.choice(constant("activities")),
    ),
    initial={},
)
```

This declaration says that `votes` is a map, every key is text, every value is a configured activity, and a new poll begins empty.

The initial state is:

```
{"votes": {}}
```

The main field types are:

```
T.boolean()
T.text()
T.number(minimum=0, maximum=100)
T.integer(minimum=1)
T.choice(options)
T.rank(options)
T.optional(T.text())
T.sequence(T.text())
T.map(T.text(), T.number())
```

Types are contracts, not comments. EDSL checks initial values and every proposed transition against them.

6. Commands

A command defines one legal kind of write:

```
"vote": Command(
    inputs={
        "voter": T.text(),
        "activity": T.choice(constant("activities")),
    },
    effects=(
        put(
            "votes",
            key=input_("voter"),
```

```

        value=input_("activity"),
        once=True,
    ),
),
)

```

The caller cannot omit an input, invent an extra one, supply a non-text voter, or submit an activity outside the configured choices.

`input_("voter")` and `input_("activity")` refer to validated command inputs. They form serializable expressions rather than reading Python variables now.

The `put` effect adds one map entry. `once=True` leaves an existing key unchanged, giving the poll first-write-wins behavior.

7. Effects and atomicity

The ordinary effects are deliberately few:

```

set_("field_name", value)
set_once("field_name", value)
put("map_field", key, value, once=False)
append("sequence_field", value)
when(condition, effect)

```

A command may contain several effects:

```

effects=(
    set_once("proposer", input_("player")),
    set_once("offer", input_("amount")),
    append(
        "history",
        record(player=input_("player"), amount=input_("amount")),
    ),
)

```

Every expression reads the same pre-command snapshot. EDSL constructs the proposed state, validates all of it, and commits everything together. If validation fails, none of the effects are committed. This is essential when interviews execute concurrently.

8. Requirements

A command can declare when it applies:

```

respond = Command(
    inputs={
        "decision": T.choice(["accept", "reject"]),
    },
    require=field("offer") != None,
    effects=(
        set_once("decision", input_("decision")),
    ),
)

```

The responder cannot act before an offer exists. An unmet requirement produces no state change.

Writes are accepted for processing atomically. Any returned command status is advisory: it is not a durable receipt or a promise that the caller possesses the newest state after other concurrent writes.

9. Views

Stored fields and participant-visible data are different concepts. A view declares exactly what a reader may observe:

```
view={
    "votes": field("votes"),
    "vote_count": field("votes").length(),
}
```

`field("votes")` refers to persistent state. The count is derived rather than stored redundantly.

Views can hide information until close:

```
from edsl.sharedstate.dsl import choose, current
```

```
view={
    "ballot_count": field("ballots").length(),
    "ballots": choose(
        current.closed,
        field("ballots"),
        {}),
}
```

They can depend on the viewer:

```
view={
    "public_price": field("price"),
    "your_private_value": choose(
        current.viewer.role == "buyer",
        field("buyer_value"),
        None),
}
```

`current` is an execution-time reference, not Jinja text. Its allowed paths are typed and permission-checked.

10. Testing without a survey

A machine is testable as an ordinary transition system:

```
state = activity_poll.initial_state()
```

```
state = activity_poll.apply(
    state,
    command="vote",
    voter="Amina",
    activity="hike",
)
```

```
state = activity_poll.apply(
    state,
    command="vote",
    voter="Boris",
    activity="sailing",
)
```

The state is now:

```
{
    "votes": {
        "Amina": "hike",
        "Boris": "sailing",
    }
}
```

Trying to change Amina's vote leaves it unchanged because `once=True`.

Render the visible data separately:

```
activity_poll.render_view(state)
{
    "votes": {
        "Amina": "hike",
        "Boris": "sailing",
    },
    "vote_count": 2,
}
```

These calls are useful for unit tests. Production execution performs the same transition through a transactional store.

11. Serialization

The machine can be serialized:

```
payload = activity_poll.to_json()
```

An input reference becomes data resembling:

```
{
    "op": "ref",
    "namespace": "input",
    "name": "activity"
}
```

The payload contains no executable Python. Serialization is why the DSL uses `field("votes")` rather than a closure and `input("activity")` rather than a lambda.

12. Creating one SharedState space

Place the machine in a state definition:

```
from edsl.sharedstate import SharedState

group_state = SharedState(
    poll=activity_poll,
)
```

One space now contains a machine named `poll`. It may contain several machines:

```
group_state = SharedState(
    poll=activity_poll,
    comments=message_log,
)
```

The poll and message log have separate fields and commands but share one scope lifecycle. No file path appears here: this defines state, not storage.

13. Creating many scopes with SharedStateMap

Most simulations need one independent space per group:

```
from edsl.sharedstate import SharedStateMap

groups = SharedStateMap(group_state)
```

Conceptually, it is a dictionary:

```
{
    "family-1": SharedState(...),
    "family-2": SharedState(...),
    "family-3": SharedState(...),
}
```

Spaces need not all be created in advance. Selecting a new key can create its initial state:

```
family_1 = groups.by("family-1")
family_2 = groups.by("family-2")
```

In a survey, selection usually comes from agent grouping metadata:

```
family = groups.by(current.group.family_id)
```

One `SharedStateMap` contains many scopes. One `SharedState` contains the named machines within a scope.

14. Connecting the poll to a survey

Define the question:

```
activity = QuestionMultipleChoice(
    question_name="activity",
    question_text=Stem(
        """
        Current votes:
        {{ poll.votes }}

        Which activity would you most prefer?
        """,
    ),
    question_options=activities,
)
```

`Stem` preserves structured question text and references through local or remote serialization. Jinja belongs inside the presentation. State commands use typed references.

Put an explicit read before the question and the write after it:

```
family = groups.by(current.group.family_id)

survey = Survey([
    family.poll.refresh(),
    activity,
    family.poll.vote(
        voter=current.agent.name,
        activity=activity.answer,
    ),
])
```

Read the sequence literally:

1. refresh this family's poll;
2. ask the question using that snapshot;
3. submit the answer as a vote.

`activity.answer` means the validated answer to this question. It is not a placeholder string.

15. Why reads are explicit

A state value rendered when a survey launches can be stale before a participant reaches the question. `family.poll.refresh()` states exactly when freshness matters.

Every refresh is logged at the **Results** level with a read ID, selected scope, machine name, snapshot ID, interview ID, and survey position. This supports auditing without promising that a snapshot remains newest after it is read.

16. Agent grouping and scheduling

Grouping is metadata associated with agents, not a secret persona trait:

```
grouping = AgentGrouping(  
    family_id={  
        "Amina": "family-1",  
        "Boris": "family-1",  
        "Chen": "family-2",  
        "Daria": "family-2",  
    },  
    turn={  
        "Amina": 1,  
        "Boris": 2,  
        "Chen": 1,  
        "Daria": 2,  
    },  
)
```

```
people = people.with_grouping(grouping)
```

The schedule consumes the same named grouping:

```
schedule = GroupedRoundRobin(  
    group_by=current.group.family_id,  
    order_by=current.group.turn,  
    max_concurrent_groups=10,  
)
```

Different families may run concurrently. Within a family, the schedule controls turn order. The state definition does not schedule interviews.

17. Storage and execution

Attach storage when executing, not when defining data:

```
job = (  
    survey  
    .by(people)  
    .using(shared_state=groups)  
    .with_schedule(schedule)  
)
```

```
results = job.run()
```

For local execution, EDSL manages a local state service. For remote execution, it creates or attaches to a remote store and sends a durable reference with the job:

```
results = job.run(remote=True)
```

Results contains state provenance:

```
results.shared_state.incoming  
results.shared_state.reads  
results.shared_state.writes  
results.shared_state.outgoing
```

Incoming and outgoing values are snapshots. A later run can continue from the outgoing state or deliberately branch from an earlier snapshot.

18. Building a log from first principles

A message log demonstrates a sequence field and `append`:

```
from edsl.sharedstate.dsl import append, record, reduce_

message_log = Machine(
    name="MessageLog",
    constants={},
    fields={
        "messages": state_field(T.sequence(T.map()), initial=[]),
    },
    commands={
        "post": Command(
            inputs={
                "author": T.text(),
                "text": T.text(),
            },
            effects=(
                append(
                    "messages",
                    record(
                        author=input_("author"),
                        text=input_("text"),
                    ),
                ),
            ),
        ),
    },
    view={
        "messages": field("messages"),
        "message_count": field("messages").length(),
        "recent_messages": reduce_(
            "tail", field("messages"), count=10
        ),
    },
)
```

The poll uses a map and put; the log uses a sequence and `append`. A few primitives produce substantially different structures.

19. Derived collection expressions

Views and transitions derive values without redundant fields:

```
values = field("forecasts").values()

mean = reduce_("mean", values)
median = reduce_("median", values)
latest = reduce_("latest_by", field("history"), field="forecaster")
```

Collections can be transformed:

```
from edsl.sharedstate.dsl import filter_items, local, map_sequence

active = filter_items(
    field("orders"),
    item="order",
    predicate=local("order").get("status") == "open",
)
```

```
)

prices = map_sequence(
    active,
    item="order",
    value_expr=local("order").get("price"),
)

```

These operations are bounded and serializable. `choose`, `and`, and `or` short-circuit so guarded lookups are safe.

20. Completion and close

A machine can say when enough information has arrived:

```
complete_when=(
    field("ballots").length()
    == constant("voter_count")
)

```

Some results are computed only at close:

```
from edsl.sharedstate.dsl import set_

close_effects=(
    set_(
        "mean",
        reduce_("mean", field("choices").values()),
    ),
)

```

Before close, a view might reveal only a submission count. After close, it can reveal choices and results. Closing is an audited atomic transition.

21. Pattern cookbook

The same small set of fields and effects recurs across many simulations. This section collects common patterns without hiding them behind new classes.

Pattern: first-write-wins

Use this when each participant gets one binding submission, such as a vote, allocation, or sealed bid:

```
fields={
    "choices": state_field(T.map(T.text(), T.text()), initial={}),
}

submit = Command(
    inputs={
        "participant": T.text(),
        "choice": T.text(),
    },
    effects=(
        put(
            "choices",
            key=input_("participant"),
            value=input_("choice"),
            once=True,
        ),
    ),
)

```

A retry with the same key is harmless. A conflicting later value is ignored.

For a single shared value rather than a map entry, use `set_once`:

```
set_once("final_decision", input_("decision"))
```

Pattern: latest-write-wins

Use ordinary `put` when participants may revise their current response:

```
put(
  "forecasts",
  key=input_("forecaster"),
  value=input_("probability"),
)
```

The map always contains the latest value. If the revision history matters, keep both a current map and an append-only history in one command:

```
effects=(
  put("latest", input_("forecaster"), input_("probability")),
  append(
    "history",
    record(
      forecaster=input_("forecaster"),
      probability=input_("probability"),
      round=input_("round"),
    ),
  ),
)
```

Both changes commit atomically.

Pattern: append-only event history

Use a sequence when every event should remain available:

```
fields={
  "events": state_field(T.sequence(T.map()), initial=[]),
}
```

```
record_event = Command(
  inputs={
    "actor": T.text(),
    "action": T.text(),
  },
  effects=(
    append(
      "events",
      record(
        actor=input_("actor"),
        action=input_("action"),
        interview=current.interview.id,
      ),
    ),
  ),
)
```

Internal metadata can remain in state while the view projects only public fields.

Pattern: expose only recent history

The full log can persist while readers receive a bounded tail:

```
view={
  "event_count": field("events").length(),
  "recent_events": reduce_(
    "tail",
    field("events"),
    count=10,
  ),
}
```

This bounds prompt size without discarding the audit history.

Pattern: an atomic multi-field transition

Use one command when fields must agree with one another:

```
make_offer = Command(
  inputs={
    "proposer": T.text(),
    "amount": T.number(minimum=0, maximum=constant("stake")),
  },
  require=field("offer") == None,
  effects=(
    set_("proposer", input_("proposer")),
    set_("offer", input_("amount")),
    append(
      "history",
      record(
        kind="offer",
        proposer=input_("proposer"),
        amount=input_("amount"),
      ),
    ),
  ),
)
```

Readers never see an amount without its proposer or a current offer without the corresponding history entry.

Pattern: a staged interaction

A later command can require an earlier stage:

```
respond = Command(
  inputs={
    "responder": T.text(),
    "decision": T.choice(["accept", "reject"]),
  },
  require=(
    (field("offer") != None)
    & (field("decision") == None)
  ),
  effects=(
    set_("responder", input_("responder")),
    set_("decision", input_("decision")),
  ),
)
```

This pattern supports ultimatum games, trust games, signaling, cheap talk, principal-agent interactions, and bilateral trade.

Pattern: terminal actions

Some actions end an interaction immediately:

```
terminal_turns = filter_items(
    field("turns"),
    item="turn",
    predicate=(
        (local("turn").get("action") == "accept")
        | (local("turn").get("action") == "walk away")
    ),
)

complete_when=terminal_turns.length() > 0
```

The interview schedule can use this completion condition to stop further turns for that scope while other groups continue.

Pattern: sealed collection and reveal

Expose only submission counts until close:

```
view={
    "submission_count": field("ballots").length(),
    "ballots": choose(
        current.closed,
        field("ballots"),
        {},
    ),
    "results": choose(
        current.closed,
        field("results"),
        None,
    ),
}
```

Then compute results atomically at close:

```
close_effects=(
    set_(
        "results",
        reduce_(
            "ranked_ballot_results",
            field("ballots"),
            candidates=constant("candidates"),
        ),
    ),
)
```

Use this for voting, sealed auctions, beauty contests, and simultaneous-move games.

Pattern: viewer-specific private information

Use typed viewer context rather than string interpolation:

```
view={
    "price": field("price"),
    "your_value": choose(
```

```

        current.viewer.role == "buyer",
        field("buyer_value"),
        None,
    ),
    "your_cost": choose(
        current.viewer.role == "seller",
        field("seller_cost"),
        None,
    ),
}

```

The validator checks whether a view is permitted to disclose each context path.

Pattern: private signal history

Store histories by participant and select only the viewer's entry:

```

viewer_history = field("revealed").get(
    current.viewer.name,
    [],
)

view={
    "your_signal_history": viewer_history,
    "your_signal": choose(
        viewer_history.length() > 0,
        viewer_history.at(viewer_history.length() - 1).get("signal"),
        None,
    ),
}

```

The guard is safe because `choose` evaluates only the selected branch.

Pattern: counters from checkbox selections

A checkbox answer may increment several configured counters atomically:

```

tally = Command(
    inputs={
        "selected": T.sequence(T.choice(constant("options"))),
    },
    effects=(
        set_(
            "counts",
            reduce_(
                "increment_keys",
                field("counts"),
                keys=input_("selected"),
            ),
        ),
    ),
)

```

Unknown keys fail validation rather than creating accidental counters.

Pattern: keep only the latest submission per participant

An append-only history can still produce a latest-value view:

```

latest = reduce_(
    "latest_by",
    field("submissions"),
    field="participant",
)

view={
    "latest": latest.values(),
    "history": field("submissions"),
}

```

This is useful for forecasts, revised rankings, and deliberative estimates.

Pattern: dynamic payoff dictionaries

When participant names become result keys at runtime, use `map_of`:

```

payoffs = map_of(
    (
        field("proposer"),
        constant("stake") - field("offer"),
    ),
    (
        field("responder"),
        field("offer"),
    ),
)

```

This produces, for example:

```
{"Amina": 70, "Boris": 30}
```

No role-specific payoff helper is required.

Pattern: claim from a capacity-limited pool

The requirement checks current availability, and the effects reserve the item and record the claim together:

```

claim = Command(
    inputs={
        "claimant": T.text(),
        "item": T.text(),
    },
    require=(
        ~field("claims").contains(input_("claimant"))
        & (field("remaining").get(input_("item"), 0) > 0)
    ),
    effects=(
        put("claims", input_("claimant"), input_("item"), once=True),
        set_(
            "remaining",
            field("remaining").with_item(
                input_("item"),
                field("remaining").get(input_("item")) - 1,
            ),
        ),
    ),
)

```

For iterative allocation across complete preference rankings, use a registered matching algorithm instead of constructing an unbounded loop in the DSL.

Pattern: several structures in one scope

Machines remain small and compose at the `SharedState` layer:

```
committee_state = SharedState(  
    agenda=agenda_machine,  
    votes=voting_machine,  
    comments=message_log,  
    forecast=forecast_machine,  
)  
  
committees = SharedStateMap(committee_state)  
committee = committees.by(current.group.committee_id)
```

Do not combine unrelated fields into one large machine merely because they share a scope.

Pattern: the same structure across many groups

Define the machine once and vary only the scope key:

```
family = families.by(current.group.family_id)  
market = markets.by(current.group.market_id)  
pair = bargaining_pairs.by(current.group.pair_id)
```

The selected space changes; the machine definition and survey remain the same.

22. Registered algorithms

Most structures use fields, commands, effects, and pure expressions. A few mechanisms require an iterative algorithm:

- LMSR market settlement;
- serial dictatorship;
- deferred acceptance;
- double-auction matching.

They use reviewed, versioned EDSL capabilities:

```
from edsl.sharedstate.dsl import algorithm  
  
close_effects=(  
    algorithm(  
        "deferred_acceptance",  
        requests=field("requests"),  
        capacities=constant("capacities"),  
        priorities=constant("priorities"),  
    ),  
)
```

An algorithm is not an arbitrary callback. It accepts typed data, has no I/O authority, runs within resource limits, and produces a state proposal that must pass validation before commit.

23. Convenience constructors—only now

The explicit activity poll is a common pattern. EDSL may offer a shorter constructor:

```
from edsl.sharedstate import recipes  
  
short_poll = recipes.register(  
    key_type=T.text(),  
    value_type=T.choice(activities),
```



```

    write="once",
)

```

This is not a new primitive or execution system. It is a factory returning an ordinary `Machine` equivalent to the `ActivityPoll` definition above.

Likewise:

```

comments = recipes.log(
    entry={"author": T.text(), "text": T.text()},
)

```

is shorthand for a machine with a sequence field and append command. Authors can serialize and inspect the returned machine:

```

print(short_poll.to_json())

```

If a convenience constructor cannot be explained by showing its expanded machine, it is too magical for this API.

24. Validation and security

Recipes are untrusted data. EDSL validates them at creation and again on the execution server:

- references point to declared constants, fields, inputs, or context paths;
- initial values and transitions satisfy recursive types;
- public views cannot read unauthorized private context;
- operators, reducers, effects, and algorithms are allowlisted and versioned;
- expression depth, collection size, state size, and execution cost are bounded;
- NaN, infinity, unsafe indexing, and division by zero fail predictably;
- the complete proposed state is checked before atomic commit.

The DSL has no imports, `eval`, reflection, unrestricted attribute access, filesystem access, network access, user callbacks, or general loops.

25. Authoring checklist

Before coding, answer:

1. What persistent values exist?
2. What type does each have?
3. What is the initial state?
4. Which commands can change it?
5. What inputs does each command accept?
6. When is each command allowed?
7. Which writes are first-write-wins?
8. What may each participant see?
9. When must state be refreshed?
10. When is the activity complete?
11. What happens at close?
12. Is a registered iterative algorithm truly necessary?

Test the machine before connecting it to a survey:

- initial and empty views;
- valid transitions and invalid inputs;
- duplicate and retried writes;
- concurrent conflicts;
- viewer-specific privacy;
- completion and repeated close;
- serialization round trips;
- state and computational limits.

Conclusion

Shared state is not a special-purpose game object or a Python callback hidden in a survey. It is a typed, serializable state machine:

`constants + fields + commands + views`

`SharedState` gives machines one concrete space. `SharedStateMap` supplies as many independent scopes as the simulation needs. Explicit survey steps read and write the selected scope. Storage and concurrency are attached later without changing the logical definition.

Once that foundation is clear, standard recipes are welcome—but only as short, inspectable constructors for machines the reader already knows how to build.